

CCF: Calcul Rapide et Évolutif des Composantes Connexes avec MapReduce

Aurélien Duvignac-Rosa, Edoardo Piciucchi, Jean-Marc Fauvel

Abstract—Identifier les composantes connexes dans un graphe est un problème bien connu dans de nombreux domaines d'application tels que l'analyse des réseaux sociaux, l'exploration de données, le traitement d'images, et bien d'autres encore. Ce projet porte sur l'étude de [1], qui vise à identifier toutes les composantes connexes dans un graphe donné à l'aide du paradigme *MapReduce*. Notre implémentation s'est faite exclusivement en *pyspark*. Nous présenterons tout d'abord notre implémentation de l'algorithme *CCF* appliqué à deux structures de données : les *RDD* (*Resilient Distributed Dataset*) et *DataFrame* pour ensuite appliquer notre implémentation sur le graphe fourni dans l'article [1] et enfin étendre notre travail sur des graphes beaucoup plus importants afin de mener une étude comparative entre les deux structures de données exploitées et les algorithmes étudiés.

Keywords—Composantes connexes ; Graphes à grande échelle ; *PySpark* ; *MapReduce*.

I. INTRODUCTION

[1] propose un algorithme de calcul de composantes connexes en utilisant le modèle de programmation *MapReduce*, basé sur l'algorithme de *Label Propagation* et conçu pour être rapide et extensif (*scalable*), c'est-à-dire pouvant être utilisé sur de grands ensembles de données et un grand nombre de machines.

L'algorithme a pour objectif d'identifier les sous-graphes connectés au sein d'un graphe G . Ce dernier devant être représenté par une collection de paires (N_1, N_2) où N_1 et N_2 sont des nœuds du graph G qui ont une connexion entre eux. Le couple (N_1, N_2) représentant une arête du graphe.

L'algorithme décrit dans [1] identifie les sous-graphes connectés en transformant leurs paires sous la forme (N, N_{min}) , où N est un nœud et N_{min} le nœud de plus petite valeur du même sous-graphe. Conçu pour traiter de grands graphes, il utilise une programmation distribuée basée sur *MapReduce*, permettant de paralléliser le traitement sur plusieurs machines.

II. OBJECTIFS DU PROJET

- 1) Lire, comprendre et expliquer l'algorithme décrit dans [1] ;
- 2) Coder l'algorithme en *Spark* en utilisant à la fois des *RDD* (*Resilient Distributed Dataset*) et *DataFrame* ;
- 3) L'implémentation doit être exécutée en Python, et optionnellement en *Scala* ;
- 4) Effectuer une analyse comparative des versions en *RDD* et *DataFrame* sur des graphes de tailles croissantes ;
- 5) Utilisation de *DataBricks*¹ pour les petits graphes, et *Google Cloud Cluster* pour les plus gros (< 20 Gb).

¹L'ensemble des graphiques présentés dans ce document ont été obtenus par *DataBricks*

III. PRÉSENTATION DE L'ALGORITHME CCF

L'algorithme *CCF*, présenté dans [1], se décompose en trois sous algorithmes distincts : *CCF-Iterate*, *CCF-Dedup* et une optimisation : *CCF* (*secondary sorting*), chacun respectivement appelé dans les algorithmes de ce document 1, 2 et 3. Ces derniers ont été étudiés et implémentés en *pyspark*.

Algorithme 1 CCF-Iterate

```
1: function MAP(key, value)
2:   emit(key, value)
3:   emit(value, key)
4: end function
5: function REDUCE(key, values)
6:    $min \leftarrow key$ 
7:   for value  $\in$  values do
8:     if value < min then
9:        $min \leftarrow value$ 
10:    end if
11:    valueList.add(value)
12:  end for
13:  if min < key then
14:    emit(key, min)
15:    for value  $\in$  valueList do
16:      if value  $\neq$  min then
17:        Counter.NewPair.increment(1)
18:        emit(value, min)
19:      end if
20:    end for
21:  end if
22: end function
```

Algorithme 2 CCF-Dedup

```
1: function MAP(key, value)
2:   temp.entity1  $\leftarrow$  key
3:   temp.entity2  $\leftarrow$  value
4:   emit(temp, null)
5: end function
6:
7: function REDUCE(key, values)
8:   emit(key.entity1, key.entity2)
9: end function
```

Algorithme 3 CCF-Iterate (with secondary sorting)

```
1: function MAP(key, value)
2:   emit(key, value)
3:   emit(value, key)
4: end function
5: function REDUCE(key, values)
6:   minVal ← values.next()
7:   if minVal < key then
8:     emit(key, minVal)
9:   end if
10:  for value ∈ values do
11:    Counter.NewPair.increment(1)
12:    emit(value, minVal)
13:  end for
14: end function
```

A. Explication des sous algorithmes

- *CCF-iterate* : cette étape consiste à propager les étiquettes des composants en mettant à jour chaque nœud avec la plus petite étiquette disponible. Le processus se répète jusqu'à stabilisation des étiquettes ;
- *CCF-dedup* : Cette étape consiste à supprimer les arêtes redondantes pour réduire le graphe et éviter des calculs inutiles ;
- *CCF-iterate (with secondary sorting)*, une variante de l'algorithme *CCF-iterate*, optimise l'utilisation de la mémoire en éliminant la nécessité de rechercher la valeur minimale dans un groupe de nœuds connectés.

B. Présentation de l'algorithme pas à pas

- **données d'entrées** : paires de nœuds (N_1, N_2) , représentant les arêtes du graphe, stockées sous forme d'un graphe noté $G = (V, E)$ avec V (vertices), les sommets et E (edges), les arêtes.
- **données de sortie** : paires de nœuds (N_i, N_{min}) où $N_i \forall i \in [0; n]_{\mathbb{N}}$, n étant le nombre de nœud du graphe complet, est un nœud du sous-graphe et N_{min} est le nœud de plus petite valeur appartenant au même sous-graphe. Ces paires sont stockées sous forme d'un graphe noté $G' = (V', E')$ avec V' les sommets et E' les arêtes.

C. Implémentation de l'algorithme CCF en paradigme Spark

L'implémentation décrite fournie dans le *notebook* associé permet de réaliser les étapes de l'algorithme *CCF* décrites dans l'exemple de [1]. Les résultats obtenus nous ont permis de constater une erreur au niveau de l'exemple de [1]. En effet, dans la figure 5.1 de [1], bien que $H > G$, l'arête $H \rightarrow G$ n'est pas transmise dans le graphe de la colonne *Reducer* lors de la première itération, ce qui propage une erreur à chaque itération. Cependant, le résultat final reste correct.

Pour le prouver, il suffit d'instancier le paramètre *debug* à *True* dans la méthode *workflow* décrite dans le code fourni. Cela permet d'afficher les graphes successifs obtenus après chaque itération de l'algorithme.

L'implémentation s'est faite en s'appuyant sur la programmation orientée objet : permettant ainsi de réduire au

maximum la redondance de code et ainsi en assurer une meilleure lisibilité. Les deux structures de données *RDD* et *DataFrame* ont été encapsulées dans deux classes distinctes : *GraphRDD* et *GraphDF* elle-mêmes héritant d'une classe mère *Graph* qui implémente à la fois les méthodes abstraites requises comme *reducer* et *mapper* mais également des méthodes communes comme *ccf-iterate* et *workflow* : en charge d'exécuter le *workflow* complet de l'algorithme de recherche de composantes connexes. De plus, l'algorithme *CCF-Iterate (secondary sorting)* (cf. algorithme 3), présenté dans [2] a été implémenté par l'intermédiaire de deux autres classes : *Graph_RDD_Secondary_Sorting* et *Key1Partitioner*.

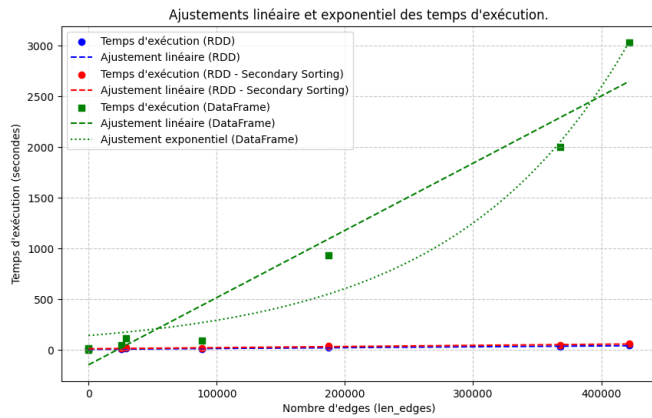
IV. PRÉSENTATION DES RÉSULTATS OBTENUS

Les résultats obtenus, présentés dans la figure 2 et le tableau 1, ont permis de mettre en exergue deux points importants. D'une part, comme en témoignent la figure 2a et le tableau 1, la structure de données *RDD* est beaucoup plus *résiliente* et permet une *scalabilité* sur des graphes de très grandes tailles à la différence de la structure *DataFrame* qui ne fournit pas un temps de traitement aussi faible que les *RDD*. Les points bleus (resp. rouges) de la figure 2a, représentent les temps d'exécution successifs pour les *RDD* avec l'algorithme *CCF* (resp. *CCF with secondary sorting*), montrent une croissance quasi-linéaire, confirmée par les lignes d'ajustement linéaire (pointillées). Cela indique que les *RDD* sont extrêmement efficaces et leur *scalabilité* est quasi constante, même pour des graphes de grande taille. Quant aux temps d'exécution des *DataFrames*, représentés par les points verts, deux éléments notables peuvent s'en dégager :

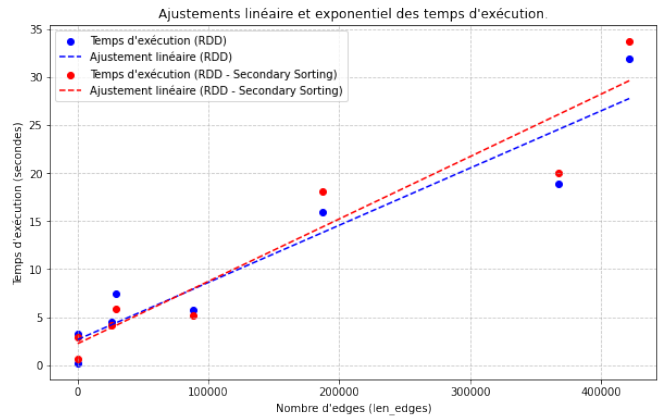
- 1) une croissance linéaire est à noter dans les premières étapes ;
- 2) Une tendance exponentielle qui se manifeste pour des graphes de grande taille, confirmée par l'ajustement exponentiel

Cela reflète un comportement beaucoup moins efficace et une incapacité à traiter efficacement des graphes de taille croissante. Cette divergence de comportement provient du fait que les *RDD* sont conçus pour un traitement distribué natif et exploitent mieux les capacités du système distribué (parallélisme, partitionnement, résilience). Tandis que les *DataFrames*, bien que plus optimisés pour des cas de calcul spécifiques, nécessitent des surcoûts liés à la gestion des métadonnées et aux optimisations internes, ce qui peut engendrer des ralentissements pour des charges de travail très importantes. La tendance exponentielle observée pour les *DataFrames* peut s'expliquer par une gestion inefficace de la mémoire et des ressources au fur et à mesure que la taille des graphes augmente. La surcharge d'optimisation et la gestion des partitions deviennent des goulets d'étranglement, entraînant une explosion des temps de calcul.

D'autre part, la figure 2b démontre que l'efficacité de l'algorithme de *secondary sorting* n'a pu être mise en évidence avec les *datasets* fournis (cf. [3]–[8]). Ces résultats sont corroborés par [1] et [9] qui indiquent que malgré des graphes très



(a) Dataset complet avec régression linéaire et exponentielle.



(b) Dataset complet avec régression linéaire (RDD et RDD secondary sorting).

Figure 2: Comparatif des temps d'exécution en fonction du nombre d'arêtes d'entrée.

conséquents, ces derniers sont trop petits pour observer une amélioration significative du temps d'exécution. Des tailles de plusieurs millions d'arêtes sont nécessaires.

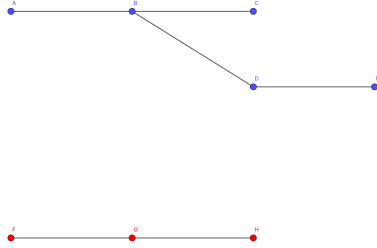


Figure 1: Graphe issu de [1].

V. GARANTIE DE L'INTÉGRITÉ DES RÉSULTATS OBTENUS

Pour garantir l'intégrité des résultats, une analyse minutieuse des éléments de *debug* du graphe fourni dans [1], illustré par la figure 1, a été menée. De plus, après chaque traitement d'un jeu de données, une comparaison des résultats obtenus pour chaque structure de données a été réalisée (cf. tableau 1). En effet, nous avons fait l'hypothèse que si le nombre d'itération, le nombre de *clusters* obtenus, et les éléments relatifs à chaque *cluster* sont identiques pour l'ensemble des structures de données exploitées, et *a fortiori* des classes et des méthodes implémentées, les résultats sont correctes.

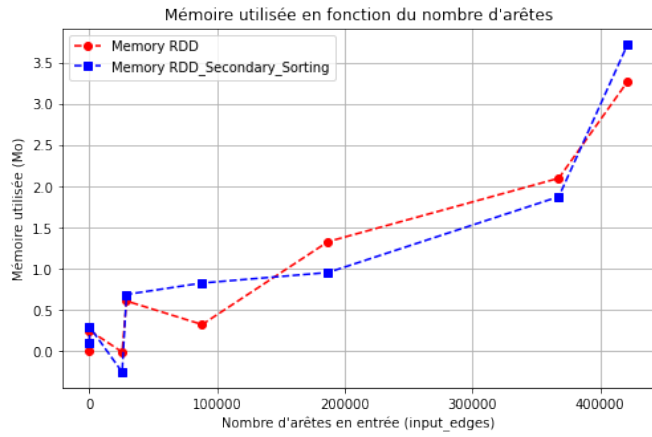
Tableau 1: Résumé des résultats pour différents *datasets* et structures de données.

dataset	SDD	nb d'arêtes d'entrée	Temps d'exécution (s)	itérations	clusters
Vide	RDD	0	0.190470	1	0
Vide	RDD Secondary Sorting	0	0.678277	1	0
Vide	Dataframe	0	1.883	1	0
[1]	RDD	6	3.230718	4	2
[1]	RDD Secondary Sorting	6	2.885972	4	2
[1]	Dataframe	6	14.630	4	2
[3]	RDD	25571	4.538241	5	1
[3]	RDD Secondary Sorting	25571	4.197027	5	1
[3]	Dataframe	25571	45.649	5	1
[4]	RDD	28980	7.400668	7	354
[4]	RDD Secondary Sorting	28980	5.816829	7	354
[4]	Dataframe	28980	111.948	7	354
[5]	RDD	88234	5.781553	5	1
[5]	RDD Secondary Sorting	88234	5.156801	5	1
[5]	Dataframe	88234	89.160	5	1
[6]	RDD	186936	15.948345	7	567
[6]	RDD Secondary Sorting	186936	18.105718	7	567
[6]	Dataframe	186936	929.319	7	567
[7]	RDD	367662	18.921314	6	1065
[7]	RDD Secondary Sorting	367662	19.994005	6	1065
[7]	Dataframe	367662	2004.748	6	1065
[8]	RDD	421578	31.950516	7	61
[8]	RDD Secondary Sorting	421578	33.771847	7	61
[8]	Dataframe	421578	3036.168	7	61

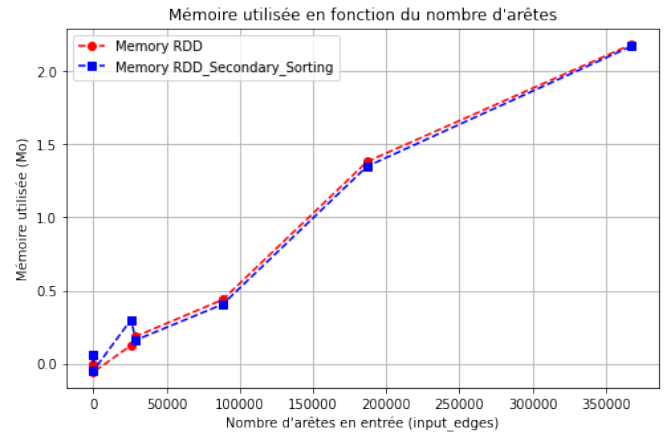
VI. ANALYSE QUANTITATIVE SUR L'UTILISATION DE MÉMOIRE DE RDD ET RDD secondary sorting

La figure 2b ne permettant pas de mettre en évidence de manière significative l'impact du tri secondaire sur le temps d'exécution, nous avons formulé l'hypothèse que son principal avantage résidait dans la réduction de la mémoire utilisée. Pour valider cette hypothèse, notre première approche a consisté à analyser la mémoire consommée en fonction du nombre d'arêtes en entrée. Cependant, nous avons été confrontés au problème du *Garbage Collector* de *PySpark*, qui, étant déclenché après chaque traitement, entraînait des valeurs négatives de mémoire utilisée et une forte variabilité des résultats, empêchant ainsi toute validation concluante, comme illustré par la figure 3a.

Pour surmonter cette difficulté, nous avons répété dix fois le traitement des *datasets* pour les classes *RDD* et *RDD_secondary_sorting*, permettant ainsi de lisser les fluctuations dues au *Garbage Collector* et d'obtenir une tendance cohérente et croissante de la mémoire utilisée en fonction du nombre d'arêtes en entrée. Cette approche nous a permis d'obtenir la figure 3b, offrant une représentation plus fidèle de l'évolution de la mémoire consommée.



(a) Quantité de mémoire utilisée selon *RDD* et *RDD secondary sorting* (1 epoch).



(b) Quantité de mémoire utilisée selon *RDD* et *RDD secondary sorting* (10 epoch).

Figure 3: Comparatif de la quantité de mémoire utilisée en fonction du nombre d'arêtes d'entrée.

VII. CONCLUSION

Ce travail a permis d'étudier et d'implémenter l'algorithme *Connected Components Finder (CCF)*, basé sur le paradigme *MapReduce*, pour identifier les composantes connexes dans un graphe. L'implémentation a été réalisée en *pyspark* avec deux structures de données distinctes: *RDD* et *DataFrame*, afin de comparer leurs performances sur des graphes de tailles croissantes.

Les résultats obtenus montrent que les *RDD* offrent des performances bien supérieures aux *DataFrames* en termes de *scalabilité* et de temps d'exécution. En effet, les *RDD* présentent une croissance quasi-linéaire des temps d'exécution, même sur des graphes de très grande taille, grâce à leur conception native pour le traitement distribué et leur gestion efficace des partitions. À l'inverse, les *DataFrames* affichent une croissance exponentielle des temps d'exécution pour les graphes plus grands, en raison de la surcharge liée à la gestion des métadonnées et des ressources.

Par ailleurs, l'optimisation par tri secondaire (*secondary sorting*) appliquée aux *RDD* n'a pas montré de gains significatifs sur les graphes testés, probablement en raison de leur taille insuffisante pour exploiter pleinement cette approche. Toutefois, cette technique pourrait se révéler bénéfique pour des graphes contenant plusieurs millions d'arêtes. Bien que le temps d'exécution soit souvent plus élevé, l'hypothèse selon laquelle le tri secondaire réduit l'empreinte mémoire a été confirmée, malgré des courbes de consommation de mémoire très proches entre la méthode classique et cette optimisation.

En conclusion, ce travail met en évidence l'intérêt d'utiliser les *RDD* pour des graphes de grande taille en raison de leur résilience et de leur *scalabilité*, tandis que les *DataFrames* montrent des limitations importantes pour ces cas d'usage. Ces résultats ouvrent la voie à des travaux futurs portant sur l'optimisation des *DataFrames* ou l'application de ces implémentations à des graphes de taille encore plus importante.

BIBLIOGRAPHIE

- [1] H. Kardes, S. Agrawal, X. Wang, and A. Sun, "Ccf: Fast and scalable connected component computation in mapreduce," in *2014 International Conference on Computing, Networking and Communications (ICNC)*, 2014, pp. 994–998.
- [2] J. Lin and C. Dyer, *Data-Intensive Text Processing with MapReduce*, ser. Synthesis Lectures on Human Language Technologies. Morgan & Claypool Publishers, 2010.
- [3] Email-eu-core dataset. Consulté le 27 décembre 2024. [Online]. Available: <https://snap.stanford.edu/data/email-Eu-core.txt.gz>
- [4] ca-grqc dataset. Consulté le 27 décembre 2024. [Online]. Available: <https://snap.stanford.edu/data/ca-GrQc.txt.gz>
- [5] Facebook combined dataset. Consulté le 27 décembre 2024. [Online]. Available: https://snap.stanford.edu/data/facebook_combined.txt.gz
- [6] ca-condmat dataset. Consulté le 27 décembre 2024. [Online]. Available: <https://snap.stanford.edu/data/ca-CondMat.txt.gz>
- [7] Email-enron dataset. Consulté le 27 décembre 2024. [Online]. Available: <https://snap.stanford.edu/data/email-Enron.txt.gz>
- [8] cit-hep dataset. Consulté le 27 décembre 2024. [Online]. Available: <https://snap.stanford.edu/data/cit-HepPh.txt.gz>
- [9] Databricks documentation. Consulté le 27 décembre 2024. [Online]. Available: <https://databricks-prod-cloudfront.cloud.databricks.com/public/4027ec902e239c93eaaa8714f173bfc/1995521040060237/795622891085557/671947465383818/latest.html>